

EXPERIMENT 3 — Process Creation and Process States

To create a parent and child process using `fork()` and observe their PID, PPID, and different process states.

`nano exp3.c`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before fork()\n");
    printf("Current PID: %d\n\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("CHILD PROCESS\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        printf("Child is sleeping...\n");

        sleep(10);
    }
}
```

```

    printf("Child process completed.\n");
}
else
{
    // Parent process
    printf("PARENT PROCESS\n");
    printf("PID : %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("Parent is waiting for child...\n");

    wait(NULL);

    printf("Parent process completed.\n");
}

return 0;
}

```

```

bharath@DESKTOP-6D34C2R:~/OSSP$ nano exp3.c
bharath@DESKTOP-6D34C2R:~/OSSP$ gcc exp3.c -o exp3
bharath@DESKTOP-6D34C2R:~/OSSP$ ./exp3
Before fork()
Current PID: 1030

PARENT PROCESS
PID : 1030
Child PID: 1031
Parent is waiting for child...
CHILD PROCESS
PID : 1031
PPID : 1030
Child is sleeping...
Child process completed.
Parent process completed.
bharath@DESKTOP-6D34C2R:~/OSSP$

```

```

bharath@DESKTOP-6D34C2R:~/OSSP$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1        0  0 11:39 ?        00:00:00 /sbin/init
root           2        1  0 11:39 ?        00:00:00 /init
root           7        2  0 11:39 ?        00:00:00 plan9 --control-socket 7
root          50        1  0 11:39 ?        00:00:00 /usr/lib/systemd/systemd
systemd+      84        1  0 11:39 ?        00:00:00 /usr/lib/systemd/systemd
root          90        1  0 11:39 ?        00:00:01 /usr/lib/systemd/systemd
root         120        1  0 11:39 ?        00:00:00 /bin/sh /usr/lib/systemd
root         124        1  0 11:39 ?        00:00:00 /usr/sbin/cron -f -P
message+     130        1  0 11:39 ?        00:00:00 @dbus-daemon --system --
root         164        1  0 11:39 ?        00:00:00 /usr/bin/python3 /usr/bi
root         176        1  0 11:39 ?        00:00:00 /usr/lib/systemd/systemd
root         181        1  0 11:39 ?        00:00:00 /usr/libexec/wsl-pro-ser
syslog       193        1  0 11:39 ?        00:00:00 /usr/sbin/rsyslogd -n -i
_chrony      227       120  0 11:39 ?        00:00:00 /usr/sbin/chronyd -n -F
_chrony      235       227  0 11:39 ?        00:00:00 /usr/sbin/chronyd -n -F
root         251        1  0 11:39 hvc0     00:00:00 /usr/sbin/agetty --nores
root         254        1  0 11:39 ?        00:00:00 /usr/bin/python3 /usr/sh
root         273        1  0 11:39 tty1     00:00:00 /usr/sbin/agetty --nores
root         340        2  0 11:39 ?        00:00:00 /init
root         341       340  0 11:39 ?        00:00:00 /init
bharath      342       341  0 11:39 pts/0    00:00:00 -bash
root         343        2  0 11:39 ?        00:00:00 login -- bharath
bharath      388        1  0 11:39 ?        00:00:00 /usr/lib/systemd/systemd
bharath      390       388  0 11:39 ?        00:00:00 (sd-pam)
bharath      418       343  0 11:39 pts/1    00:00:00 -bash
root        1040        90  0 11:47 ?        00:00:00 (udev-worker)
root        1041        90  0 11:47 ?        00:00:00 (udev-worker)
bharath     1047       342  0 11:47 pts/0    00:00:00 ps -ef

```

What are we doing?

We use `fork()` to create a child process.

Parent

|

`fork()`

|

Child

Then we display:

- PID
- PPID
- Process state

Your practical specifically asks for PID, PPID, and process-state observations.

OSSP Practical Experiments

What is PID?

PID means **Process ID**.

Every process has its own number.

Example:

Parent PID = 1143

Child PID = 1144

Think of PID as the **ID number of a process**.

What is PPID?

PPID means **Parent Process ID**.

Example:

Child PID = 1144

Child PPID = 1143

This means:

Process 1143 is the parent of process 1144.

Process states

A process can be in different states:

Ready

The process is ready to run.

Running

The CPU is currently running the process.

Waiting

The process is waiting for something.

Terminated

The process has finished.

We use:

Bash

ps

Bash

top

and:

/proc

to observe processes.

Remember:

PID = Process ID

PPID = Parent Process ID